

PC2 – DESARROLLO SOFTWARE

Tema: Implementación de patrones de diseño Creacionales

Nombre: Erick Joel Sánchez Longa

1. Patron Singleton

Definición: Como vimos en clase el patrón Singleton garantiza que exista una sola instancia de una clase en todo el programa. Por eso se usó en la conexión a base de datos

Se implemento en:

Clase: ConexionBaseDatosGarantiza

```
1 class ConexionBaseDatos:
2     _instancia = None
3
4     def __new__(cls):
5         if cls._instancia is None:
6             cls._instancia = super().__new__(cls)
7             MONGO_URI = os.getenv(
8                 "MONGO_URI",
9                 "mongodb+srv://erickslonga24_db_user:ltg9xCcFwERsCsJL@todolistcluster.hrygcsc.mongodb.net/?appName=TodoListCluster"
10            )
11             cls._instancia.cliente = MongoClient(MONGO_URI)
12             cls._instancia.base = cls._instancia.cliente["incidencias_db"]
13             cls._instancia.incidencias = cls._instancia.base["incidencias"]
14             cls._instancia.notificaciones = cls._instancia.base["notificaciones"]
15         return cls._instancia
16
17     def verificar(self):
18         self.cliente.admin.command("ping")
19
20 bd = ConexionBaseDatos()
```

¿Qué hace este patrón?

Garantiza que solo exista una única instancia de la conexión a MongoDB Atlas en toda la aplicación, evitando saturar la base de datos con múltiples conexiones innecesarias.

2. Factory Method

Definición: Vimos que el patrón Factory Method delega la creación de objetos a un método especial llamado "factory". En lugar de usar new directamente, llamamos a un método que decide qué objeto crear.

Se implemento en:

Clase: ObservadorFactory

```

1 class ObservadorFactory:
2     @staticmethod
3     def crear_observador(tipo: str) -> ObservadorBase:
4         if tipo == "bitacora": return ObservadorBitacora()
5         elif tipo == "correo": return ObservadorCorreo()
6         else: raise ValueError(f"Tipo de observador desconocido: {tipo}")
7
8 class SistemaNotificaciones:
9     def __init__(self):
10         self._observadores: List[ObservadorBase] = []
11
12     def suscribir(self, obs: ObservadorBase):
13         self._observadores.append(obs)
14
15     def notificar(self, evento: str, datos: dict):
16         for obs in self._observadores:
17             obs.actualizar(evento, datos)
18
19 notificador = SistemaNotificaciones()
20 notificador.suscribir(ObservadorFactory.crear_observador("bitacora"))
21 notificador.suscribir(ObservadorFactory.crear_observador("correo"))
22

```

¿Qué hace este patrón?

Utilizamos el patrón factory en la creación de los observadores (Bitácora, Correo). Esto es para que si en el futuro se quiere agregar notificaciones por SMS o WSP, solo tendríamos que modificar esta fábrica sin tocar el notificador principal.

```

notificador = SistemaNotificaciones()
notificador.suscribir(ObservadorFactory.crear_observador("bitacora"))
notificador.suscribir(ObservadorFactory.crear_observador("correo"))

```

En esta parte: `ObservadorFactory.crear_observador("correo")`, usamos el patrón factory para decidir qué objeto crear.

En vez de hacer: `ObservadorCorreo()`

Directamente, hacemos: `crear_observador("correo")`

Entonces el patrón factory decide qué clase instanciar.

3. Builder

Definición: El patrón Builder sirve para construir objetos complejos paso a paso. Se usa cuando un objeto tiene muchos atributos o configuraciones y no queremos crear constructores enormes.

Se implementa en:

Clase: `IncidenciaBuilder`

Separa paso a paso la construcción del documento/diccionario de la incidencia, como la asignación de metadatos, estructuración de ubicaciones y ciudadanos, limpiando el endpoint de registro.

```
1 class IncidenciaBuilder:
2     def __init__(self):
3         self._documento = {}
4
5     def set_datos_basicos(self, codigo: str, categoria: str, descripcion: str, direccion: str, prioridad: str):
6         self._documento.update({
7             "codigo_incidencia": codigo,
8             "categoria": categoria,
9             "descripcion": descripcion,
10            "direccion": direccion,
11            "estado": "pendiente",
12            "prioridad": prioridad
13        })
14        return self
15
16    def set_ubicacion(self, lat: float, lng: float):
17        self._documento["ubicacion"] = {"latitud": lat, "longitud": lng}
18        return self
19
20    def set_ciudadano(self, nombres: str, correo: str, telefono: str):
21        self._documento["ciudadano"] = {"nombres": nombres, "correo": correo, "telefono": telefono}
22        return self
23
24    def set_metadatos(self):
25        self._documento.update({
26            "fecha_registro": datetime.now().isoformat(),
27            "fecha_actualizacion": None,
28            "medios": [],
29            "historial": []
30        })
31        return self
32
33    def build(self) -> dict:
34        return self._documento
```